

Verifier-Grounded Generate-Verify-Repair for Intent→Configuration NetOps Tasks: A Paired Mininet Emulation Study

Autonomous AI Researcher
CNSM 2026 Agentic AI Researcher Track

Abstract—We preregistered and executed a provenance-traced paired confirmatory study (N = 200 intents; 400 paired episodes) to evaluate whether a deterministic verifier-grounded generate-verify-repair (GVR) loop reduces operationally detectable policy-violation errors when a hosted LLM produces device configuration sequences from operator intent in a Mininet emulation. Execution used shared_initial_candidate semantics (one sampled initial generation per intent reused by both baseline and guarded). Baseline success (no detected policy violation) = 196/200 (98%); guarded success = 200/200 (100%). Paired risk difference (guarded - baseline) = 0.02; preregistered exact McNemar two-sided p = 0.125 (primary confirmatory test). An exploratory paired bootstrap (seed = 1729, 10,000 resamples) yields a 95% percentile CI = [0.005, 0.04]. Four verifier-triggered repairs occurred (total hosted model calls = 204), giving mean extra model calls per intent = 0.02 and mean extra calls per avoided violation = 1.0. All reported numeric values, provider-call accounting, validator traces, and analysis artifacts are drawn verbatim from the archived execution and analysis bundle. We interpret the preregistered confirmatory result conservatively and discuss construct, internal, and external validity limits and operational implications.

I. INTRODUCTION

Automating intent→configuration translation is an active NetOps research direction because natural-language or intent descriptions are a natural operator interface but can be transformed incorrectly by large language models (LLMs). Mechanically checkable faults introduced by LLMs—syntactic errors, topology/reachability violations, and ACL/policy mistakes—can lead to availability loss or exposure if applied to devices without validation [1], [4]. Generate-Verify-Repair (GVR) patterns pair an LLM generator with a deterministic verifier: the verifier checks mechanizable constraints and, if violations occur, issues localized diagnostics used to request corrective edits from the model [2], [3]. Prototype work across code and domain-specific program generation suggests verifier-grounded repair reduces mechanically verifiable errors; however, questions remain about scaling such pipelines to heterogeneous multi-vendor template families, the concrete hosted-LLM cost per avoided violation, and how to interpret paired comparisons when sampling semantics reuse the initial model candidate.

Research question and contribution. Our preregistered research question is: Does a verifier-grounded generate-verify-repair loop significantly reduce operationally detectable policy-violation errors produced by a hosted LLM when

generating network configurations for operator intents across heterogeneous multi-vendor template families and topology patterns in a Mininet-based emulation? Contributions: (1) a provenance-traced confirmatory paired experiment (C1-GVR-multivendor-2026-v1) with shared_initial_candidate semantics executed on N = 200 intents (400 episodes); (2) audited provider and verifier accounting showing repair costs and concrete hosted-model calls; (3) artifact-grounded statistical inference (exact McNemar test) and exploratory bootstrap interval (seed = 1729, 10,000 resamples); (4) a focused analysis of failure modes, threats to validity, and operationally grounded recommendations consistent with archived artifacts.

II. RELATED WORK

Recent literature on LLMs for NetOps and AIOps converges on two observations that are directly relevant to verifier-grounded pipelines. First, surveys and domain reviews document rapid uptake of LLMs for network tasks while highlighting recurring gaps in evaluation practice: missing provenance, limited verifier integration, and scarce multi-vendor or emulation-scale evidence [5], [6]. These reviews synthesize heterogeneous reports (research prototypes, bench demonstrations, and applied surveys) and conclude that while LLMs reduce human effort for drafting configurations, the community lacks standardized, provenance-traced benchmarks that pair generators with deterministically auditable verifiers. This limits cross-study comparability and weakens claims about operational safety or generalization across vendors and topologies [5], [6].

Second, targeted configuration studies and benchmarks show feasibility but also expose common failure modes. Prior NetOps evaluations (for example, work investigating LLM requirements for correct router configuration synthesis and the NetConfEval benchmark) document frequent classes of mechanically checkable failures—syntax/grammar mistakes, incorrect sequencing that violates workflow invariants, and topology/reachability errors when configurations contradict the network state [1], [4]. These studies typically evaluate generator accuracy on curated tasks and emphasize the need for automated checking; however, many prototype benchmarks either omit a closed-loop repair stage or do not provide provenance traces linking initial outputs, verifier diagnostics, and any applied repairs. In practice, reported improvements in such studies are therefore difficult to attribute to deterministic

validator interventions versus differences in prompting or sampling.

A cross-domain body of work motivates verifier-grounded mitigation architectures (generate–verify–repair). Recent methodological proposals and prototype demonstrations in code and domain-specific program generation argue that a verifier capable of expressing mechanizable constraints plus localized diagnostic feedback enables targeted repairs that reduce mechanically detectable errors [2], [3]. These works provide conceptual and single-case empirical support: they show that when a verifier can (i) deterministically check the artifact against explicit constraints and (ii) return actionable, localized diagnostics, a bounded repair policy can converge failing samples to valid artifacts. Importantly, these prior results also identify the core dependency of GVR effectiveness on verifier coverage: if a verifier cannot encode an error class, that class remains a residual risk even with repeated repair attempts [2], [3].

How our study relates. Building on the empirical observations and methodological proposals above, our experiment addresses a concrete gap: scalable, provenance-traced paired evaluations that measure the marginal effect of deterministic verification and bounded repair across heterogeneous template families and topology patterns. Unlike prior NetOps benchmarks that either evaluate single-pass generation or lack explicit provenance between generator outputs and verifier actions [1], [4], or like GVR proposals that focus on single-domain demonstrations [2], [3], our paired design (with `shared_initial_candidate` semantics) isolates the verifier/repair contribution by reusing the identical initial LLM sample for both conditions and then measuring the final guarded outcome after permissive, verifier-triggered repair. This combination—multi-vendor synthetic templates exercised in Mininet, deterministic verifier traces, provable provider-call accounting, and preregistered paired inference—directly responds to the literature’s call for provenance and auditable assessments of verifier-assisted generation [5], [6]. At the same time, we adopt the caution emphasized in the surveyed and prototype literature: verifier expressivity bounds the classes of errors that can be detected and repaired, and mechanical verification cannot by itself guarantee absence of higher-order semantic misinterpretations outside the verifier’s rule set [2], [3], [5].

III. METHODOLOGY

Preregistration and primary estimand. The study was preregistered as C1-GVR-multivendor-2026-v1 prior to any model calls (preregistration SHA-256: 9e0fc992665835975b01289f7841926ef6abf3584252c5d0459ba0ecf166bb50). The primary estimand is the `paired_success_rate_difference_guarded_minus_baseline`: the per-intent difference in the probability that an intent yields zero operationally detectable violations under the guarded pipeline versus baseline. The preregistered confirmatory test is an exact two-sided McNemar test at $\alpha = 0.05$. Predeclared exploratory summaries include a paired bootstrap percentile CI (`bootstrap_seed = 1729`; `resamples = 10,000`)

and descriptive hosted-model cost metrics (mean extra calls per intent and mean extra calls per avoided violation).

Sampling design, deterministic task generation, and strata. Task indices ($N = 200$ unique intents) were selected deterministically by the adapter before any model interactions. The sampling plan was stratified across three synthetic vendor-template families and four topology patterns (12 strata) to exercise template heterogeneity and workflow policy variety. Task payloads were generated by deterministic `netops_task_generator_v5` and include explicit per-task constraints (`allowed_touched_fields`, `workflow_policies`, `required_sequence`) and a canonical `expected_final_state` used by the deterministic verifier. The deterministic task generation and the fixed `task_index_profile` ensure that no post-hoc selection or data-dependent task filtering affected the confirmatory analysis.

`Shared_initial_candidate` semantics and paired condition definitions. Execution used `shared_initial_candidate` semantics as preregistered and archived: for each intent exactly one initial sampled generation was produced (the shared initial candidate) and that same initial text was presented to both baseline and guarded scoring/validation pipelines. Under these semantics: (a) baseline outcome equals the deterministic validator’s verdict (`validation_before`) on the shared initial candidate; (b) guarded outcome equals the final guarded-pipeline verdict after deterministic validation and any permitted repair(s). The preregistration explicitly declared that independent constrained-prompt arms were not executed; therefore any paired discordance can only arise from deterministic validator behavior and from the allowed repair call(s), not from differing first-pass samples or prompt variants.

Model configuration, sampling notation, and audited provider budget. The declared model (as recorded in the archived `model_configuration.json`) is `gpt-5-mini` (provider recorded as `openai_responses`). The archived `model_configuration.json` records `temperature = null/unset`; `null/unset` is not evidence of deterministic hosted-provider sampling and does not imply `temperature = 0`. The adapter enforced the preregistered model-call budget: `initial_generation_calls_per_task = 1`, `maximum_repair_calls_per_task = 1`, `maximum_model_calls = 400`, `planned_episode_count = 400`. Provider audit fields in the execution manifest (`hosted_model_call_event_count`, `stage_counts`, `condition_counts`, `cache_hit/miss counts`, and `cross_condition_cache_key_reuse_observed`) were used to reconcile actual model calls and to verify that cross-condition cache reuse did not occur.

Deterministic verifier architecture and scoring rule. The `deterministic_netops_validator_v5` is a rule-based verifier combining multiple mechanizable checks applied against the Mininet emulated state and the per-task payload: (1) syntactic parsing and command pattern matching to detect malformed or unrecognized commands; (2) required-command sequencing and workflow policy checks that ensure the produced sequence can satisfy `required_sequence` constraints and group/ordering invariants; (3)

reachability/topology checks that simulate or reason over the emulated final_state to detect unsafe shutdowns or simultaneous outages; and (4) ACL/policy checks encoded from task-specific policy constraints. For each evaluated candidate the verifier returns a structured validator_trace containing normalized_configuration, parsed_command_count, required_command_count, observed_touched_field_count, violation_codes (if any), violation_count, and boolean valid. The scoring rule full_intent_constraint_validation yields score = 1 (success) only when violation_count == 0 and all required mechanizable constraints are satisfied.

Repair policy and repair-prompt formation. The guarded pipeline permits up to one verifier-triggered repair call per intent and only issues a repair call when the verifier reports one or more violations on the shared initial candidate. Repair prompts are deterministic templates that include: the verifier’s localized diagnostics (violation_codes and short natural-language diagnostics), the normalized shared initial_configuration, the task’s required_sequence and workflow constraints, and a request for a corrected configuration sequence that preserves unspecified state per the task payload. Repair prompts do not alter the initial prompt template family; they request corrective edits against the shared_initial_candidate. When a repair response is returned, the verifier is re-applied; if validation_after reports no violations the guarded outcome is success. Repair Provider traces and repair_prompt SHA references are archived for each repaired pair.

Provider-call accounting and stage_counts interpretation. All provider call accounting reported in the execution manifest and deterministic_reconciliation was reconciled deterministically. The audited hosted_model_call_event_count = 204 decomposes as 200 shared_initial_generation events (one per intent) plus 4 repair calls (repair stage). The execution manifest stage_counts field records {"shared_initial_generation": 200, "repair": 4}. The condition_counts mapping in the manifest (shown as {"shared": 200, "guarded": 4}) corresponds to these stage counts under the shared_initial_candidate semantics: the label "shared" enumerates the initial sampled generations produced for reuse, while the label "guarded" in the manifest reflects the number of repair-stage calls actually invoked by the verifier. The manifest also records cache_hit_count = 0 and cross_condition_cache_key_reuse_observed = false, supporting that the same shared initial candidate was deterministically reused by both baseline and guarded conditions rather than created by cross-condition cache reuse.

Scoring decisions, exclusions, and missingness treatment. Scoring decisions followed the preregistered contract: ambiguous verifier outputs (those flagged by deterministic ambiguity rules) were conservatively treated as violations in the primary analysis. Exclusion rules predeclared duplicate tasks (detected by deterministic prompt+context hashing), unrecoverable container/template substitution failures, and infrastructure integrity failures; excluded pairs would be reported separately and included in sensitivity analyses. The primary analysis used the 'complete_pair_primary'

treatment: only complete pairs (both episodes scored) were included. The preregistered sensitivity analysis ('complete_pair_primary_with_failure_as_zero_sensitivity') treats unrecoverable or aborted pairs conservatively as failures; no such exclusions occurred in the archived run (complete_pair_count = 200; failed_episode_count = 0).

Analysis procedures and bootstrap caution. Primary inferential analysis is the exact two-sided McNemar test on the paired binary success indicators (baseline vs guarded). Secondary exploratory summaries include an empirical paired bootstrap percentile CI for the paired risk difference (bootstrap_seed = 1729, resamples = 10,000) and descriptive statistics for model-call costs (mean extra calls per intent, mean extra calls per avoided violation). Because paired bootstrap percentile intervals for binary paired differences can be unstable when the total number of discordant pairs ($n_{10} + n_{01}$) is small, we treat the bootstrap CI as exploratory magnitude information only and emphasize the exact McNemar p-value as the preregistered confirmatory outcome. Analysis code, seeds, and resampling parameters are archived and referenced in analysis_log.jsonl.

IV. RESULTS

Execution and audited provider accounting. The preregistered run completed as planned: planned_episode_count = 400; completed_episode_count = 400; complete_pair_count = 200; failed_episode_count = 0. Deterministic provider accounting (execution manifest) reports hosted_model_call_event_count = 204 decomposed into 200 shared_initial_generation events and 4 repair calls; cache_hit_count = 0; cache_miss_count = 204; stage_counts = {"shared_initial_generation": 200, "repair": 4}. These audit counts are reported verbatim from the archived execution manifest.

Paired outcomes and confirmatory test. The paired 2×2 contingency (analysis/paired_contingency_table.csv) is: $n_{11} = 196$ (both succeed), $n_{10} = 4$ (guarded succeed, baseline fail), $n_{01} = 0$ (guarded fail, baseline succeed), $n_{00} = 0$ (both fail). Baseline success rate = $196/200 = 0.98$; guarded success rate = $200/200 = 1.0$; paired risk difference (guarded - baseline) = 0.02. The preregistered exact McNemar two-sided test yields $p = 0.125$ (primary confirmatory result; do not reject at $\alpha = 0.05$).

Exploratory interval and bootstrap caution. The archived exploratory paired bootstrap (bootstrap_seed = 1729; resamples = 10,000) produced a 95% percentile CI for the paired risk difference = [0.005, 0.04]. We report this interval as exploratory and caution that bootstrap percentile intervals can be unstable when discordant pair counts ($n_{10} + n_{01}$) are small; the exact McNemar test remains the formal preregistered inference. The bootstrap seed and resampling count are archived (analysis_log.jsonl).

Repair events and cost accounting. Four verifier-triggered repairs were invoked across 200 intents. Total hosted model calls = 204 (200 shared initial + 4 repairs), giving mean hosted model calls per intent = 1.02 and mean extra calls per intent due to repairs = 0.02. Each repair corrected one baseline failure in the archived contingency; thus mean extra calls per

avoided violation = 1.0. These arithmetic derivations follow directly from the audited `provider_call` counts in deterministic reconciliation and the execution manifest.

Representative artifacts and substantiation. Representative scoring artifacts (execution/scoring/task-000004-*, task-000005-*, task-000006-*) show identical `shared_initial_candidate` content and identical `shared_initial` responses reused by both baseline and guarded conditions. For the four discordant pairs, `validator_trace.validation_before` recorded detected violations and `validation_after` recorded that a repair was applied and the final configuration passed `full_intent_constraint_validation`. The raw `shared_initial` response files, validator traces, and repair decisions are archived and cited in the execution manifest; they substantiate the contingency counts and repair accounting above.

Power and interpretation. The preregistered power analysis targeted a 10 percentage point absolute reduction with $N \approx 157$ (McNemar approximation); $N = 200$ was chosen to permit stratified descriptions. The observed baseline failure prevalence (2%) was lower than some planning scenarios, reducing power to detect small absolute improvements and producing the small discordant count (4) that underlies the non-significant McNemar result. We therefore interpret the $p = 0.125$ as the definitive confirmatory outcome under the preregistered design and treat the bootstrap CI as supplementary magnitude information.

V. DISCUSSION

Causal interpretation under `shared_initial_candidate` semantics. Because execution used `shared_initial_candidate` semantics (one sampled initial generation per intent reused by baseline and guarded), paired improvements arise solely from deterministic validator/repair actions. The preregistration explicitly declared that independent constrained-prompt arms were not executed; thus the observed $n_{10} = 4$ discordant pairs reflect validator-triggered repairs that converted failing `shared_initial` candidates into passing final configurations. Any statements attributing improvements to prompt-engineering or differing first-pass samples would be incorrect for this run.

Provider accounting and manifest labels. The archived execution manifest and deterministic reconciliation provide audited `provider_call` counts that clarify how model events map to pipeline stages. The manifest records `hosted_model_call_event_count = 204` and decomposes that total into `stage_counts = {"shared_initial_generation": 200, "repair": 4}`. Condition or stage labels observed in the manifest (for example entries labeled "shared":200 and "guarded":4) reflect this decomposition: the 200 "shared_initial_generation" events correspond to the single initial sampled generation produced and reused for both baseline and guarded episodes, while the 4 repair events are additional guarded-stage model calls that occurred only when the deterministic verifier reported violations. This audited decomposition underpins the cost accounting reported in Results (mean extra model calls per intent = 0.02; mean extra calls per avoided violation = 1.0) and supports the causal claim that paired gains derive

from verifier-triggered repairs rather than from independent re-sampling.

Statistical interpretation and bootstrap caution. The paired bootstrap percentile CI we report (95% CI = [0.005, 0.04]; `bootstrap_seed = 1729`; `resamples = 10,000`) is explicitly exploratory. Small discordant pair totals ($n_{10} + n_{01} = 4$ here) make bootstrap percentile intervals sensitive to resampling variability and to the discrete structure of paired binary data; consequently such intervals can underrepresent sampling uncertainty or be unstable in edge cases. For this reason we rely on the preregistered exact McNemar two-sided test ($p = 0.125$) as the formal confirmatory inference and present the bootstrap interval only to provide a complementary magnitude estimate archived with its seed and resampling count. Researchers planning follow-ups should (a) anticipate discordant-count regimes when designing paired tests, (b) consider planning for larger N or higher baseline failure prevalence to increase discordant counts, and (c) report both exact conditional inferences and multiple interval estimators to better triangulate effect size in sparse discordance settings.

Verifier coverage and residual semantic risks. deterministic_netops_validator_v5 checks syntactic correctness, required sequencing, reachability against the emulated state, and encoded ACL/policy constraints derived from task payloads. These mechanizable checks were sufficient to detect and localize the failures corrected by the four repairs. However, the verifier can only detect violations that are expressible in its rule set and encoded policies; higher-order semantic misinterpretations of intent that do not manifest as a mechanizable constraint violation remain possible false negatives. The observed complete elimination of mechanically detectable baseline violations by the bounded repair budget applies only to the verifier's detectable fault classes; extending verifier expressivity (for example, formal policy ontologies or richer semantic specifications) is necessary to reduce residual semantic risk.

Operational implications and bounded generality. Within the constrained Mininet emulation, synthetic template families, and validator rule set used here, a small repair budget removed all mechanically detectable baseline violations at very low hosted-model cost. This artifact-grounded observation suggests that, in settings where a deterministic verifier can express relevant constraints and produce localized, actionable diagnostics, a bounded GVR policy can eliminate template-driven mechanically checkable faults in similar emulated contexts. We emphasize bounded generality: these results do not establish production readiness, multi-model generality, or coverage for semantic errors outside the verifier. Re-deploying the approach in operational environments requires (i) richer verifier coverage tuned to production policies, (ii) paired designs that also execute independent multi-sample initial generations to separate sampling/prompting effects from repair effects, and (iii) replication across multiple hosted models and more realistic hardware testbeds.

Recommendations for future evaluations. Based on the archived evidence and the statistical constraints observed,

we recommend that future confirmed-design studies: (1) preregister whether `shared_initial_candidate` semantics or independent-sample semantics will be used and plan estimands accordingly; (2) select sample sizes or baseline scenarios that anticipate sufficient discordant pairs for confirmatory power, or else use study designs that enlarge expected discordance (e.g., harder task strata); (3) broaden verifier expressivity to capture more semantic/policy classes and measure verifier false-negative rates against annotated ground truth; and (4) report audited `provider_call` decompositions and seeds for bootstrap/replication so others can unambiguously reconcile cost and resampling diagnostics. These recommendations are directly motivated by patterns in the archived execution manifest, deterministic reconciliation, and analysis artifacts.

VI. LIMITATIONS

- Scope limited to Mininet-style emulation with synthetic, provenance-traced intent→configuration tasks; results do not generalize directly to production hardware or live operations.
- Single declared model (gpt-5-mini) evaluated; multi-model generality is not established.
- Deterministic validator coverage limited to mechanically checkable errors (syntax, reachability, ACL/policy encodings); higher-level semantic or specification misinterpretations outside the validator may remain undetected.
- `shared_initial_candidate` semantics imply observed paired differences derive only from verifier-triggered repair; this design does not measure effects of alternative prompting strategies or independent multi-sample candidate selection.
- Observed confirmatory effect (2 percentage points) did not reach statistical significance at $\alpha = 0.05$ for $N = 200$ (exact McNemar $p = 0.125$); low baseline failure prevalence (2%) reduced power to detect small absolute improvements under some preregistered scenarios.

VII. CONCLUSION

We preregistered and executed a provenance-traced paired confirmatory study (C1-GVR-multivendor-2026-v1) using `shared_initial_candidate` semantics to measure whether a deterministic verifier plus bounded repair reduces mechanically detectable policy violations for intent→configuration tasks in a Mininet emulation. The preregistered exact McNemar two-sided test did not reject at $\alpha = 0.05$ ($p = 0.125$). Guarded GVR invoked four repairs and corrected the four observed baseline violations at low model-call overhead (model calls = 204; mean extra calls per intent = 0.02), yielding a paired risk difference estimate of 0.02 and an exploratory paired bootstrap CI = [0.005, 0.04] (bootstrap_seed = 1729, resamples = 10,000). These artifact-grounded results indicate that when a deterministic verifier can express and check relevant constraints and provide localized feedback, a small repair budget can eliminate mechanically detectable policy violations in similar template-based emulated settings. The results do not establish production readiness or multi-model generality. All

reported numbers, provider accounting, validator traces, and analysis artifacts are drawn verbatim from the archived execution and analysis bundles referenced in the preregistration and execution manifests.

References [1] R. Mondal, A. Tang, R. Beckett, T. Millstein, and G. Varghese, "What do LLMs need to Synthesize Correct Router Configurations?", *Proceedings of the ACM*, 2023, doi: 10.1145/3626111.3628194. [2] R. Cadenas, "Convergent Correctness in Stochastic Code Generation: A Generate-Verify-Repair Architecture for Deterministic Validation of Autonomous AI Coding Agents in Regulated Environments", *SSRN*, 2026, doi: 10.2139/ssrn.6754899. [3] Z. Ding, "Executable but Wrong: Verifier-Grounded Contracts and Counterexamples for LLM-Generated Music Programs", 2026, doi: 10.31224/7995. [4] C. Wang, M. Scazzariello, A. Farshin, S. Ferlin, D. Kostić, and M. Chiesa, "NetConfEval: Can LLMs Facilitate Network Configuration?", *Proceedings of the ACM*, 2024, doi: 10.1145/3656296. [5] S. Y. Long et al., "A Survey on Intelligent Network Operations and Performance Optimization Based on Large Language Models", *IEEE Communications Surveys & Tutorials*, 2025, doi: 10.1109/comst.2025.3526606. [6] L. Zhang et al., "A Survey of AIOps in the Era of Large Language Models", *Proceedings of the ACM*, 2025, doi: 10.1145/3746635.

DISCLOSURE STATEMENT

Declared model (archived): gpt-5-mini (provider recorded as `openai_responses`). Adapter family: `hosted_netops_gvr_v1`. Deterministic validator: `deterministic_netops_validator_v5`. Task generator: `deterministic_netops_task_generator_v5`. Preregistration: C1-GVR-multivendor-2026-v1 (SHA-256 = 9e0fc992665835975b01289f7841926ef6abf3584252c5d0459ba0ecf166bb50), recorded prior to any model calls. Initial master prompt archived at `provenance/master_prompt.txt` (SHA-256 = 1872df1e1805d2d96940456ca016bd665d1d5196add77f5acd1582bb39b15ba). Human role: a human Research Director selected the topic family and constrained the initial master prompt prior to the autonomy lock; after the lock the AI autonomously performed literature verification, preregistration, experiment execution, analysis, peer review, revision, and manuscript drafting; no human manually edited technical manuscript content after the autonomy lock. Sampling semantics: execution used `shared_initial_candidate` (exactly one sampled initial generation per intent was produced and reused by both baseline and guarded); archived model configuration records `temperature = null/unset` (`null/unset` is not evidence of deterministic provider sampling and does not imply `temperature = 0`). Provider accounting (audited): `hosted_model_call_event_count = 204` decomposed as 200 shared initial generations + 4 repair calls. Reported numbers, provider-call accounting, validator traces, and analysis artifacts are drawn verbatim from the archived execution and analysis bundles referenced in the preregistration and execution manifests.

REFERENCES

- [1] Rajdeep Mondal, Alan Tang, Ryan Beckett, Todd Millstein, George Varghese, "What do LLMs need to Synthesize Correct Router Configurations?", 2023, 10.1145/3626111.3628194
- [2] Rafael Cadenas, "Convergent Correctness in Stochastic Code Generation: A Generate-Verify-Repair Architecture for Deterministic Validation of Autonomous AI Coding Agents in Regulated Environments", 2026, 10.2139/ssrn.6754899
- [3] Zhengyang Ding, "Executable but Wrong: Verifier-Grounded Contracts and Counterexamples for LLM-Generated Music Programs", 2026, 10.31224/7995
- [4] Changjie Wang, Mariano Scazzariello, Alireza Farshin, Simone Ferlin, Dejan Kostić, Marco Chiesa, "NetConfEval: Can LLMs Facilitate Network Configuration?", 2024, 10.1145/3656296
- [5] S.Y. Long, Jingjing Tan, Bomin Mao, Fengxiao Tang, Yangfan Li, Ming Zhao, Nei Kato, "A Survey on Intelligent Network Operations and Performance Optimization Based on Large Language Models", 2025, 10.1109/comst.2025.3526606
- [6] Lingzhe Zhang, Tong Jia, Mengxi Jia, Yifan Wu, Aiwei Liu, Yong Yang, Zhonghai Wu, Xuming Hu, Philip S. Yu, Ying Li, "A Survey of AIOps in the Era of Large Language Models", 2025, 10.1145/3746635